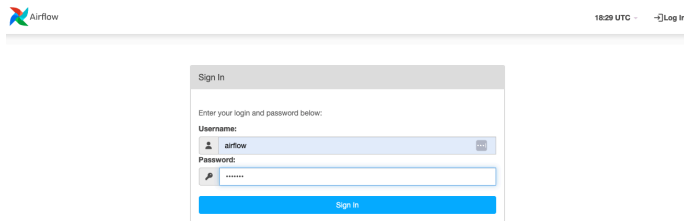


Airflow - Advanced Concepts

Open URL <http://108.129.216.179:8080/> (port 8080 of VM IP 108.129.216.179). Login to Airflow UI using default username airflow and password airflow



Airflow 18:29 UTC - [?] Log In

Sign In

Enter your login and password below:

Username:
airflow

Password:
airflow

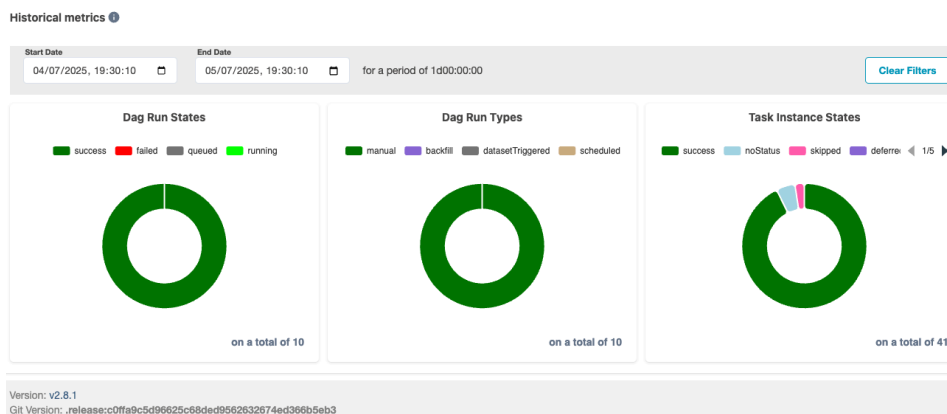
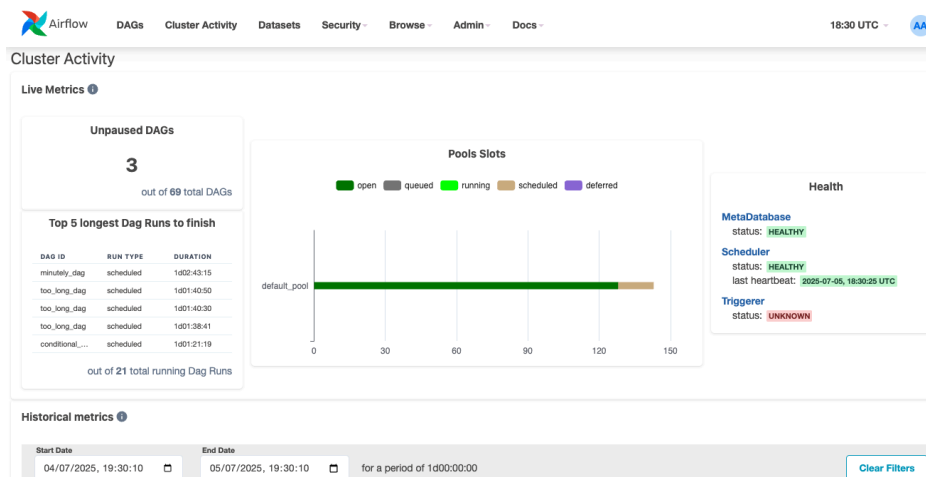
Sign In

Cluster Monitoring:
Navigate to Admin -> Cluster Activity



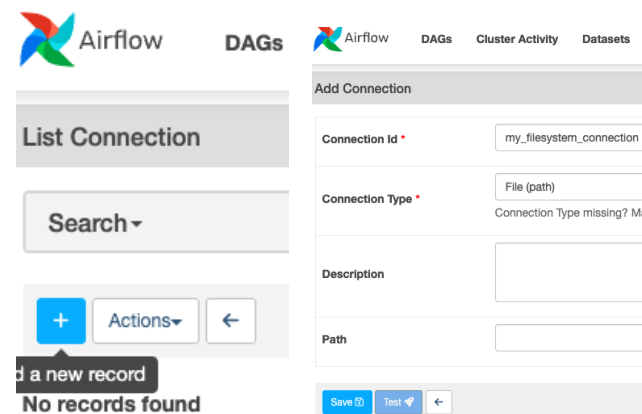
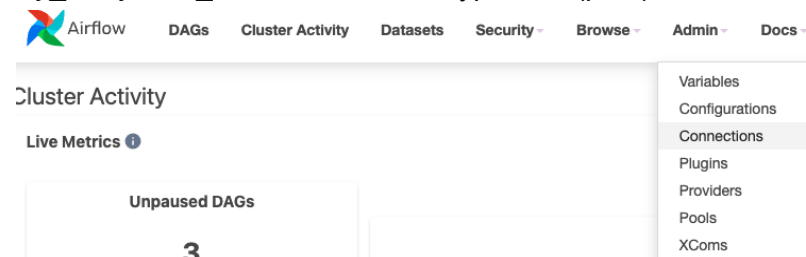
DAGs

Observe Live Metrics and scrolling down the page Historical Metrics etc



Sensors and Connections:

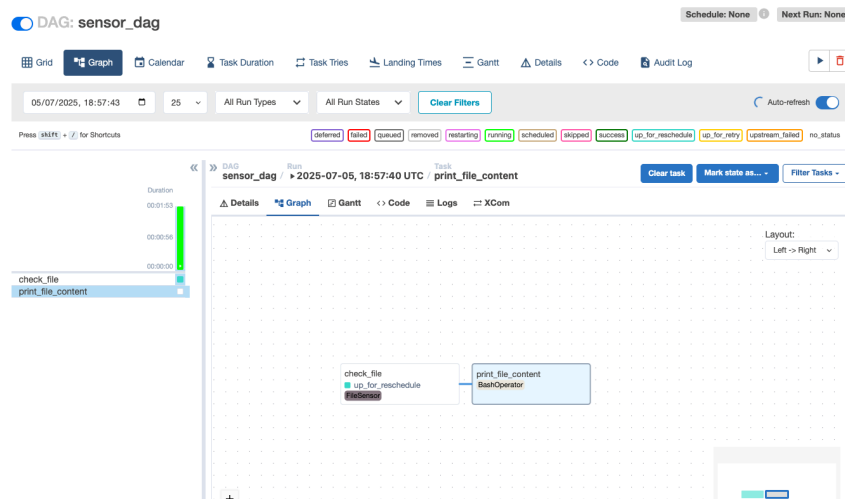
Navigate to Admin -> Connection and select the + option. Fill in details Conn Id: *my_filesystem_connection* Conn Type: *File (path)* and click *Save* button



Create a *sensor_dag.py* file in *dags/*

```
sensor_dag.py U X
tascientest-data-engineering > airflow > dags > sensor_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.sensors.filesystem import FileSensor
4 from airflow.operators.bash import BashOperator
5
6
7 with DAG(
8     dag_id = "sensor_dag",
9     schedule_interval = None,
10    tags = ["tutorial", "datascientest"],
11    start_date = days_ago(0)
12 ) as dag:
13     my_sensor = FileSensor(
14         task_id = "check_file",
15         fs_conn_id = "my_filesystem_connection",
16         filepath = "my_file.txt",
17         poke_interval = 30,
18         timeout = 5 * 30,
19         mode = "reschedule"
20     )
21
22     my_task = BashOperator(
23         task_id = "print_file_content",
24         bash_command = "cat /tmp/my_file.txt",
25     )
```

Trigger sensor_dag in Airflow UI. It will go *UP_FOR_RESCHEDULE* while the file is missing

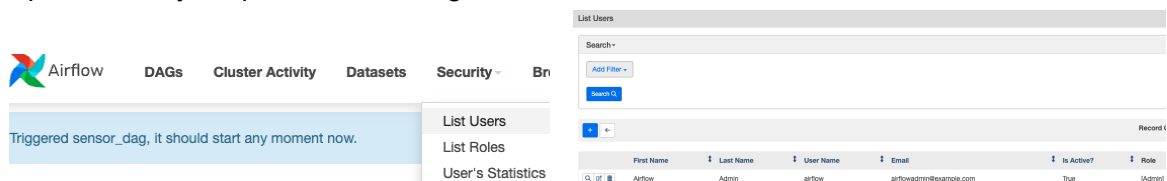


Run the command below. File may be found now and the task should succeed

```
ubuntu@ip-172-31-46-112:~/datascientest-data-engineering/airflow/dags$ docker exec -it airflow_airflow-worker_1 bash
default@35a3cfe351c7:/opt/airflow$ echo "hello world" >> /tmp/my_file.txt
default@35a3cfe351c7:/opt/airflow$
```

User Management:

Open Security dropdown and navigate to List Users



Click + to add a new user and select the Save button. Logout and login with new user details

The screenshot shows the Airflow web interface with the Add User form and the Sign In form. The Add User form has fields for First Name, Last Name, User Name, Email, Role, Password, and Confirm Password. The Sign In form has fields for Username and Password.

As user daniel with limited permissions, observe inability to trigger DAGs

DAGs										
All 70	Active 2	Paused 68	Running 20	Failed 4	Filter DAGs by tag	Search DAGs	Auto-refresh			
DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links		
☒ decorator_xcom_dag data scientist tutorial	airflow	1	None	2025-07-05, 13:27:47		2				
☒ sensor_dag data scientist tutorial	airflow	2	None	2025-07-05, 19:03:44		1	1			

Logout and back in again with default username airflow and password airflow again

Documentation:

Create a documented_dag.py file in dags/

```
documented_dag.py x
tascientest-data-engineering > airflow > dags > documented_dag.py
1 from airflow import DAG
2 from airflow.utils.dates import days_ago
3 from airflow.operators.python_operator import PythonOperator
4 import time
5
6
7 def sleep_1_sec():
8     time.sleep(1)
9
10 with DAG(
11     dag_id = "documented_dag",
12     doc_md = """
13     # Documented DAG
14     This `DAG` is documented and the next line is a quote:
15
16     > Airflow is nice
17
18     This DAG has been made:
19
20     * by DataScientest
21     * with documentation
22     * with caution
23     """,
24     start_date = days_ago(0),
25     schedule_interval = None
26 ) as dag:
27
28     task1 = PythonOperator(
29         task_id = "sleep1",
30         python_callable = sleep_1_sec,
31         doc_md = """
32         # Task1
33         Task that is used to sleep for 1 sec
34         """,
35     )
36
37     task2 = PythonOperator(
38         task_id = "sleep2",
39         python_callable = sleep_1_sec,
40         doc = """
41         Task 3
42         It has an ugly description.
43         """,
44     )
45
46 task1 >> task2
```

Return to Airflow UI and search for documented_dag. Observe the documentation directly on the DAG documented_dag in DAGs menu

🔍 DAG: documented_dag

Schedule: NoneNext Run: None

GridGraphCalendarTask DurationTask TriesLanding TimesGanttDetailsCodeAudit Log

DAG Docs

Documented DAG

This `DAG` is documented and the next line is a quote:

Airflow is nice

This DAG has been made:

- by DataScientest
- with documentation
- with caution

Trigger DAG by clicking the play button. It is also possible to view task documentation via Graph View and selecting a task and then Instance Details

🔍 DAG: documented_dag

Schedule: False

GridGraphCalendarTask DurationTask TriesLanding TimesGanttDetailsCodeAudit Log

Task Instance: sleep1 @ 2025-07-05, 19:30:42

Task Instance DetailsRendered TemplateLogXCom

Task Instance Details

Dependencies Blocking Task From Getting Scheduled

Dependency	Reason
Dagrun Running	Task instance's dagrun was not in the 'running' state but in the state 'success'.
Task Instance State	Task is in the 'success' state.

Attribute: doc_md

Task1

Task that is used to sleep for 1 sec

Task Instance Attributes

Attribute	Value
custom_operator_name	None
dag_model	<DAG: documented_dag>
dag_run	<DagRun documented_dag @ 2025-07-05 19:30:42.872664+00:00: manual_2025-07-05T19:30:42.872664+00:00, state:success, queued_at: 2025-07-05 19:30:42.900827+00:00, externally triggered: True>

Runtime environment and pools:
Navigate to Admin -> Pools. Create a pool micro_pool with slots 1

Airflow

DAGsCluster ActivityDatasetsSecurityBrowseAdminDo

DAG: documented_dag

GridGraphCalendarTask DurationTask TriesLanding TimesGanttDetailsCodeAudit Log

VariablesConfigurationsConnectionsPluginsProvidersPoolsXComs

Added Row

List Pool

Search

Actions

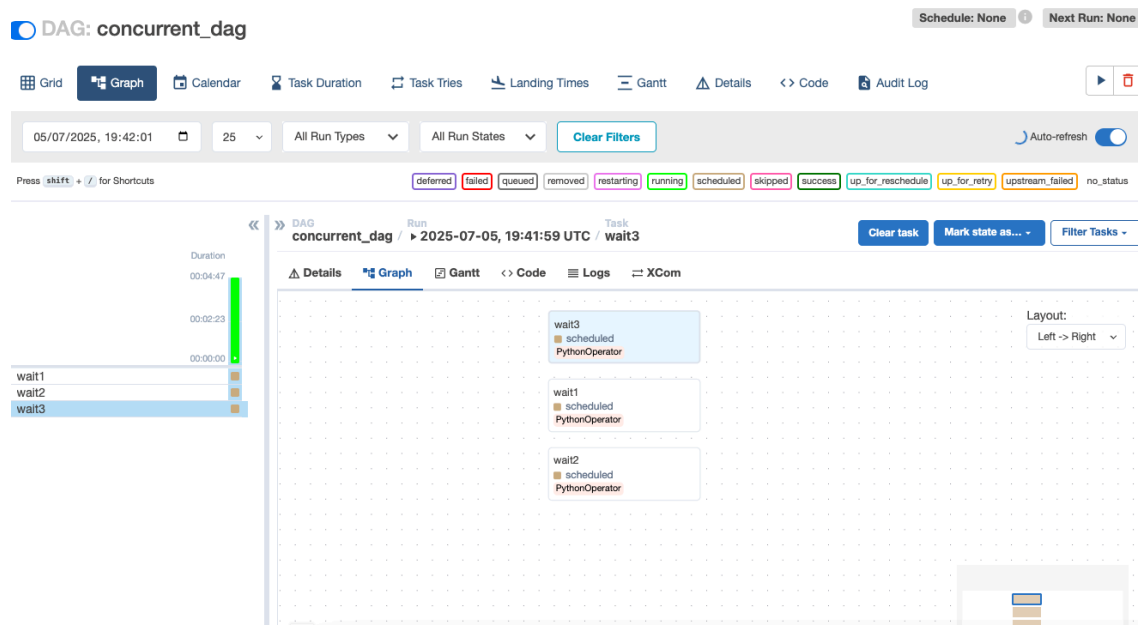
Record Count: 2

	Pool	Description	Slots	Running Slots	Queued Slots	Scheduled Slots	Deferred Slots
☐	admin_pool		1	0	0	0	0
☐	default_pool	Default pool	128	0	0	15	0

Create a concurrent_dag.py file in dags/

```
concurrent_dag.py U X
atascientest-data-engineering > airflow > dags > concurrent_dag.py
1 from airflow import DAG
2 from airflow.operators.python_operator import PythonOperator
3 from airflow.utils.dates import days_ago
4 import time
5
6
7 def wait_1_minute():
8     time.sleep(60)
9
10 with DAG(
11     dag_id = "concurrent_dag",
12     schedule_interval = None,
13     start_date = days_ago(0),
14     default_args = {
15         "pool": "micro_pool"
16     }
17 ) as dag:
18     task1 = PythonOperator(
19         task_id = "wait1",
20         python_callable = wait_1_minute,
21     )
22
23     task2 = PythonOperator(
24         task_id = "wait2",
25         python_callable = wait_1_minute,
26     )
27
28     task3 = PythonOperator(
29         task_id = "wait3",
30         python_callable = wait_1_minute,
31     )
```

Open Airflow UI, search for concurrent_dag and trigger it. Notice that tasks run one after the other to roughly 3 minutes in total



Comment out the `default_args` in `concurrent_dag.py`. Save the file.

```
concurrent_dag.py U X
atascientest-data-engineering > airflow > dags > concurrent_dag.py
1 from airflow import DAG
2 from airflow.operators.python_operator import PythonOperator
3 from airflow.utils.dates import days_ago
4 import time
5
6
7 def wait_1_minute():
8     time.sleep(60)
9
10 with DAG(
11     dag_id = "concurrent_dag",
12     schedule_interval = None,
13     start_date = days_ago(0),
14     # default_args = {
15     #     "pool": "micro_pool"
16     # }
17 ) as dag:
18     task1 = PythonOperator(
19         task_id = "wait1",
20         python_callable = wait_1_minute,
21     )
22
23     task2 = PythonOperator(
24         task_id = "wait2",
25         python_callable = wait_1_minute,
26     )
27
28     task3 = PythonOperator(
29         task_id = "wait3",
30         python_callable = wait_1_minute,
31     )
```

Re-trigger the DAG `concurrent_dag`. Now all three tasks run in parallel, roughly 1 min in total

